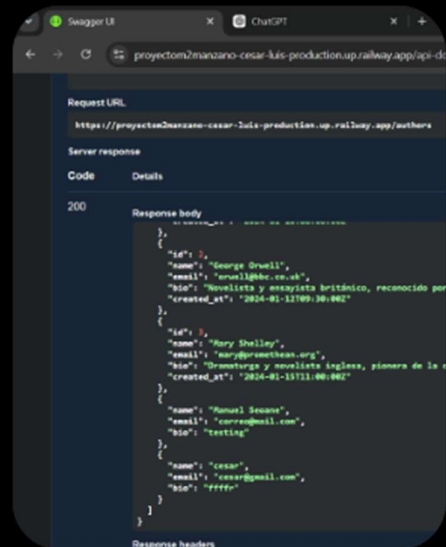




modulo2(2).txt
Documento

por que cuando ingreso los datos el id y created_at no se agregan de manera automatica

entonces necesitas que la tabla **authors** tenga algo parecido a:

SQL

```
CREATE TABLE authors (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL,
  bio TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Y después el controlador debería hacer el `INSERT` en PostgreSQL, por ejemplo:

```
JavaScript

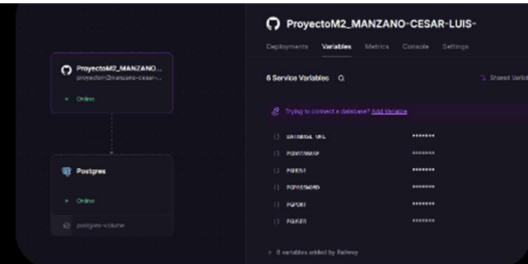
const postaddauthor = async (req, res) => {
  try {
    const { name, email, bio } = req.body;

    const result = await pool.query(
      `INSERT INTO authors (name, email, bio)
      VALUES ($1, $2, $3)
      RETURNING *`,
      [name, email, bio]
    );

    res.status(201).json({
      msg: "Autor agregado correctamente",
      data: result.rows[0]
    });

  } catch (error) {
    console.error(error);

    res.status(500).json({
      msg: "Error al crear el autor"
    });
  }
};
```



el problema para mi esta en el rayliway por que mi codigo si funciona de 10 cuando lo pruebo localmente:

Sí, tu sospecha tiene bastante sentido. Si localmente `GET /comments` funciona perfectamente y en Railway devuelve `500`, entonces hay que revisar principalmente la conexión y el estado de la base de datos de Railway, no tu controlador.

Y en tu captura veo algo importante: tu servicio de Node tiene las variables `DATABASE_URL`, `PGDATABASE`, `PGHOST`, `PGPASSWORD`, `PGPORT` y `PGUSER`, así que las variables sí están llegando al servicio.

Pero que las variables existan no significa que la base de datos de Railway tenga las mismas tablas/datos que tu PostgreSQL local.

La prueba que nos va a sacar de dudas

Entrá en:

Railway → PostgreSQL → Console

y ejecutá:



```
-- Crear la base de datos
CREATE DATABASE blog_db;
-- Crear usuario específico para la aplicación
CREATE USER blog_user WITH PASSWORD 'blog_password_2026';
-- Conectar a la base de datos
\c blog_db
-- Dar permisos al usuario en la base de datos
GRANT ALL PRIVILEGES ON DATABASE blog_db TO blog_user;
-- Dar permisos en el schema public
GRANT ALL PRIVILEGES ON SCHEMA public TO blog_user;
Ver más ▾
```

Perfecto. Ahora veo exactamente qué pasó. **Tu script local no se puede ejecutar completo tal cual en Railway**, porque allí la base de datos `railway` ya existe y Railway administra el usuario y la conexión.

La buena noticia es que **no necesitás crear otra base de datos ni otro usuario**. Solo tenemos que crear las tablas dentro de la base de datos de Railway.

1. No ejecutes estas partes en Railway

No ejecutes:

SQL

```
CREATE DATABASE blog_db;
```

4. Cargamos los datos

Después podés ejecutar tus inserts.

Autores

SQL

```
INSERT INTO authors (name, email, bio) VALUES
('Ana García', 'ana@example.com', 'Desarrolladora full-stack apasionada por Node.js'),
('Carlos Ruiz', 'carlos@example.com', 'Escritor técnico especializado en bases de datos'),
('María López', 'maría@example.com', 'Ingeniera de software con foco en APIs REST');
```

Posts

SQL

```
INSERT INTO posts (title, content, author_id, published) VALUES
('Introducción a Node.js', 'Node.js es un runtime de JavaScript...', 1, true),
('PostgreSQL vs MySQL', 'Ambas bases de datos tienen ventajas...', 2, true),
('APIs RESTful', 'REST es un estilo arquitectónico...', 1, true),
('Manejo de errores en Express', 'El manejo apropiado de errores...', 3, false),
('Async/Await explicado', 'Las promesas simplifican el código asíncrono...', 1, false);
```

Y podemos agregar comentarios

Como ahora ya existen autores y posts:

SQL

```
INSERT INTO comments (content, author_id, post_id) VALUES
('Excelente explicación de Node.js.', 2, 1),
('Muy buen artículo.', 3, 1),
('Me resultó muy útil la comparación.'. 1, 2):
```

+ Pregunta lo que quieras